Code Explanation

Code Explanation: AWS Glue PySpark Job Unit TestsThis is a comprehensive test suite for an AWS Glue data pipeline. Below are the clear steps explaining what the code does:---

1. Module Setup & Imports

- **Purpose**: Import necessary testing and PySpark libraries- **Key imports**: - `pytest` for test framework - `pyspark.sql` for Spark DataFrame operations - `unittest.mock` for mocking AWS services - Custom `SDLCWizardPipeline` class from the main job module---

2. Test Fixtures (Reusable Test Components)

2.1 Spark Session Fixture

- Creates a local Spark session for testing- Configured with 2 cores and 2 shuffle partitions- Automatically stops after all tests complete

2.2 Pipeline Fixture

- Instantiates the `SDLCWizardPipeline` class- Mocks the S3 client to avoid real AWS calls- Returns a configured pipeline object for each test

2.3 Sample Data Fixtures

Three fixtures create test datasets:**a) `sample_customer_data`**- 5 customer records with complete, valid data- Includes: customer_id, name, email, phone, address, city, state, zip, registration date, status**b) `sample_order_data`**- 7 order records linked to customers- Includes: order_id, customer_id, order_date, amount, status, product_id, quantity, payment_method**c) `sample_customer_data_with_nulls`**- 5 customer records with data quality issues: - NULL customer_id - String "Null" as a name - NULL email - Duplicate record---

3. Test Cases (20 Tests Total)

Tests 1-2: Initialization & Access

Test 1: `test_initialize_spark_contexts`- Verifies Spark, Glue, and S3 contexts initialize correctly**Test 2: `test_validate_s3_access`**- Mocks S3 client and verifies bucket access validation works---

Tests 3-4: Data Ingestion (FR-INGEST-001)

Test 3: `test_read_customer_data`- Writes sample customer data to temporary CSV file- Reads it back using pipeline's `read_data_safe` method- Verifies 5 records loaded with correct columns**Test 4: `test_read_order_data`**- Same pattern for order data- Verifies 7 records loaded with correct columns---

Tests 5-7: Data Cleaning (FR-CLEAN-001)

Test 5: `test_clean_null_values`- Uses dataset with NULL values in critical fields- Calls `clean_data` method- Verifies records with NULLs are removed- Confirms only 1 clean record remains after removing NULLs and duplicates**Test 6: `test_clean_null_strings`**- Tests removal of string literals "Null", "NULL", "null"- Verifies no such strings remain in cleaned data**Test 7: `test_remove_duplicates`**- Verifies duplicate customer_id records are removed- Confirms no customer_id appears more than once---

Tests 8-10: SCD Type 2 Implementation (FR-SCD2-001)

Test 8: `test_add_scd2_columns`- Adds SCD2 tracking columns: IsActive, StartDate, EndDate, OpTs- Verifies new records have IsActive=True**Test 9: `test_scd2_upsert_new_records`**- Tests inserting completely new records- Verifies all records marked as active**Test 10: `test_scd2_upsert_changed_records`**- Changes email for customer C001- Performs SCD2 upsert- Verifies: - Old record marked inactive (IsActive=False) - New record created and marked active - Total of 2 records for C001 (historical tracking)---

Tests 11-12: Business Logic

Test 11: `test_calculate_customer_aggregate_spend` (FR-AGG-001)- Joins customer and order data- Calculates per-customer metrics: - `total_spend`: Sum of all order amounts - `order_count`: Number of orders - `avg_spend_per_order`: Average order value- Verifies C001 has total_spend=350 (150+200) and order_count=2**Test 12: `test_generate_order_summary` (FR-SUMMARY-001)**- Generates order summary by customer: - Total orders - Total amount - Average order amount - First and last order dates- Verifies C001 has 2 orders---

Test 13: Data Writing

Test 13: `test_write_data_safe`- Writes DataFrame to temporary location as Parquet- Reads it back and verifies record count matches---

Test 14: End-to-End Integration

Test 14: `test_end_to_end_pipeline`- **Most comprehensive test** - runs entire pipeline: 1. Writes sample data to temp input locations 2. Configures pipeline with temp paths 3. Mocks S3 validation and Hudi writes 4. Executes full `run_pipeline()` method 5. Verifies all outputs created: - Customer catalog with SCD2 columns - Order catalog - Customer aggregate spend - Order summary- Tests all FRD requirements in one flow---

Test 15: Error Handling

Test 15: `test_error_handling`- Tests invalid S3 paths return None/False- Verifies graceful failure without exceptions---

Tests 16-18: Schema & Configuration Validation

Test 16: `test_get_customer_schema`- Validates customer schema has all 10 required fields from TRD**Test 17: `test_get_order_schema`**- Validates order schema has all 8 required fields from TRD**Test 18: `test_configuration_validation`**- Verifies pipeline configuration matches TRD: - S3 source paths - S3 output paths - Glue database name - SCD2 column names---

Tests 19-20: Data Quality Checks

Test 19: `test_data_quality_customer`- Verifies no NULL customer_ids- Validates all emails contain "@" symbol**Test 20: `test_data_quality_order`**- Verifies all order amounts are positive- Confirms no NULL order_ids or customer_ids---

4. Test Execution

The `if __name__ == "__main__"` block runs pytest with:- Verbose output (`-v`)- Short traceback format (`--tb=short`)---

Summary

This test suite provides **comprehensive coverage** of:- **Data ingestion** from S3- **Data cleaning** (NULLs, duplicates, invalid values)- **SCD Type 2** historical tracking- **Business aggregations** (customer spend, order summaries)- **Data quality** validation- **Error handling** and configuration validation- **End-to-end pipeline** executionAll tests are isolated, repeatable, and use temporary data to avoid side effects.